Industrial Internship Report on

"Banking Information System"

Prepared by

Nageshwaran C

R.M.K. Engineering College

Domain: Core Java | Duration: 4 Weeks

Internship at UniConverge Technologies Pvt Ltd (UCT)

TABLE OF CONTENTS

1. Preface

This report presents a summary of my 4-week industrial internship undertaken in the Core Java domain, carried out under the guidance of UniConverge Technologies Pvt Ltd (UCT) in association with upskill Campus (USC). The internship provided an opportunity to move beyond classroom learning and apply Core Java concepts to a real-world software project.

During this internship, I worked on developing a "Banking Information System" — a console-based Java application for managing customer accounts, transactions, and banking operations, backed by a relational database. Over the course of four weeks, the project evolved from a basic in-memory application to a robust system integrated with JDBC, MySQL, JUnit testing, and industry-standard design patterns such as Singleton and Factory.

An internship of this nature is an essential step in career development, as it bridges the gap between academic knowledge and industry expectations. It builds practical coding discipline, exposes students to real project constraints, and develops soft skills such as collaboration, communication, and time management — all of which are difficult to acquire through classroom instruction alone.

The program was structured on a weekly basis, with clear goals set at the start of each week, hands-on project development through the week, and a progress report submitted at the end covering achievements, challenges faced, learning resources used, and goals for the following week. This structured approach ensured steady, measurable progress and provided regular opportunities for mentor feedback.

Over these four weeks, I gained hands-on experience in Core Java, Object-Oriented Programming, Collections, Exception Handling, JDBC, SQL, JUnit testing, and design patterns, while also improving my problem-solving and debugging abilities. The overall experience was highly rewarding and has strengthened both my technical foundation and my confidence in working on real-world software projects.

I would like to thank my mentors and the team at UniConverge Technologies Pvt Ltd and upskill Campus for their continuous guidance and constructive feedback throughout the internship, and my team members for their collaboration and support during project development.

To my juniors and peers who will undertake this internship in the future: approach every task with curiosity, don't hesitate to ask questions, and focus on understanding concepts deeply rather than just completing tasks — the real learning happens when you apply what you study directly to your project.

2. Introduction

2.1 About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) is a company established in 2013, working in the Digital Transformation domain and providing industrial solutions with a prime focus on sustainability and return on investment (RoI).

To develop its products and solutions, UCT leverages cutting-edge technologies such as the Internet of Things (IoT), Cyber Security, Cloud Computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, and front-end development.

Some of UCT's key platforms and solution areas include:

- UCT Insight — an IoT platform for quick deployment of IoT applications, built using Java (backend) and ReactJS (frontend), supporting MySQL and various NoSQL databases.
- Factory Watch — a Smart Factory platform providing production and asset monitoring, OEE, and predictive maintenance solutions.
- LoRaWAN-based Solutions — for Agritech, Smart Cities, Industrial Monitoring, and Smart Utility Metering.
- Predictive Maintenance — Industrial machine health monitoring using Embedded Systems, Industrial IoT, and Machine Learning.

2.2 About upskill Campus (USC)

upskill Campus, along with The IoT Academy and in association with UniConverge Technologies, facilitated the smooth execution of the complete internship process. USC is a career development platform that delivers personalized executive coaching in an affordable, scalable, and measurable way.

2.3 Objective

The objectives of this internship program were to:

- Get practical, hands-on experience of working in the software industry.
- Apply Core Java concepts to solve a real-world project problem.
- Build a working, end-to-end banking application using industry-standard practices.
- Improve understanding of Object-Oriented Programming, JDBC, and design patterns.
- Develop personal skills such as communication, collaboration, and problem-solving.

2.4 Reference

[1] Oracle Java Documentation — https://docs.oracle.com/en/java/

[2] Project GitHub Repository — https://github.com/NAGESH192/upskillcampus.git

[3] Weekly Progress Reports, Weeks 1–4, Core Java Internship, UCT/USC

2.5 Glossary

- JDBC (Java Database Connectivity) — API for connecting Java applications to relational databases.
- OOP (Object-Oriented Programming) — programming paradigm based on classes, objects, and inheritance.
- DAO — Data Access Object, a pattern used to separate database access logic from business logic.
- JUnit — a Java testing framework used to write and run repeatable automated tests.
- Singleton Pattern — a design pattern that restricts a class to a single shared instance.
- Factory Pattern — a design pattern used to create objects without specifying the exact class to instantiate.

3. Problem Statement

Traditional record-keeping for banking operations, when done manually or through simple ad-hoc scripts, is error-prone, difficult to scale, and lacks structured validation, auditability, and secure data persistence. The assigned problem statement was to design and develop a "Banking Information System" — a Java-based application that allows a bank to manage customer accounts and transactions programmatically, with proper validation, error handling, and persistent storage, while following sound object-oriented design principles.

The system needed to support core banking operations such as account creation (savings and current accounts), deposits, withdrawals, balance inquiry, fund transfer between accounts, account closure, and a complete, queryable transaction history — all while maintaining data integrity and following industry-relevant design and coding practices.

4. Existing and Proposed solution

Existing solutions in this space range from full-scale commercial core-banking software (which are complex, closed-source, and not suitable for learning purposes) to simple in-memory academic examples that lack persistence, validation, or realistic design patterns. Most beginner-level implementations store data only in memory, without database persistence, structured exception handling, or automated testing — making them unsuitable for real-world use.

The proposed solution is a modular, console-based Java application designed with clean object-oriented principles:

- Interface-driven design using Transactable and Auditable interfaces to define contracts for account behaviour.
- Separate account types (SavingsAccount, CurrentAccount) created dynamically using the Factory design pattern.
- A Singleton-managed database connection layer to ensure a single, thread-safe connection instance.
- Persistent storage of customer and transaction data in a MySQL database via JDBC, replacing in-memory storage.
- Custom exception classes (e.g., InsufficientFundsException, InvalidAccountException) for clear, structured error handling.
- JUnit test cases covering all major modules to validate correctness and maintain code quality.

The value addition of this solution lies in demonstrating a realistic, production-style architecture — including database integration, design patterns, and automated testing — within a scope that remains understandable and extensible for learning purposes.

Code submission (Github link)

https://github.com/NAGESH192/upskillcampus.git

Report submission (Github link)

https://github.com/NAGESH192/upskillcampus.git

5. Proposed Design/ Model

The Banking Information System follows a layered, modular design that separates the user interface, business logic, and data persistence layers, making the system easier to maintain, test, and extend.

5.1 High Level Diagram

At a high level, the system consists of four major components:

- Console UI Layer — a menu-driven interface for users to perform banking operations.
- Business Logic Layer — Account Management, Transaction Processing, and Customer Data Handling modules.
- Data Access Layer — JDBC-based DAO classes that communicate with the MySQL database using PreparedStatements.
- Database Layer — MySQL relational database storing customer, account, and transaction records.

5.2 Low Level Diagram

At the class level, the design includes:

- Transactable and Auditable interfaces implemented by account classes to define standard operations and audit behaviour.
- SavingsAccount and CurrentAccount classes, instantiated through an AccountFactory.
- A Singleton DatabaseConnectionManager class controlling the single shared JDBC connection.
- Custom exception classes: InsufficientFundsException and InvalidAccountException.
- Use of Collections (ArrayList, HashMap) for in-memory caching of transaction history and fast customer lookup, alongside persistent database storage.

5.3 Interfaces

The system interfaces with a MySQL database through JDBC using PreparedStatements for all queries (SELECT, INSERT, UPDATE, DELETE), with try-with-resources used throughout to ensure connections and statements are safely closed. Data flows from the console UI, through the business logic layer, to the DAO/data access layer, and finally to the database, with results returned back up the same path for display to the user.

6. Performance Test

Since this application handles financial data, correctness, data integrity, and reliability were the key constraints considered, rather than raw speed. The main constraints identified were: correctness of business logic (balances, transfers), data consistency during concurrent operations, and proper handling of invalid input/edge cases.

6.1 Test Plan/ Test Cases

JUnit test cases were written to cover the major modules: Account Management, Transaction Processing, and Customer Data Handling. Test cases included normal-flow scenarios (deposit, withdrawal, transfer) as well as edge cases (insufficient funds, invalid account numbers, and boundary values).

6.2 Test Procedure

Tests were implemented using JUnit 5, using @Test, @BeforeEach, and @AfterEach annotations, along with assertion methods such as assertEquals, assertThrows, and assertTrue. Fund transfer operations were tested for atomicity, verifying that failed transfers correctly rolled back without leaving the system in an inconsistent state.

6.3 Performance Outcome

By the end of Week 4, JUnit test cases covered approximately 75% of the codebase, with a target of exceeding 90% coverage by project completion. Core operations — account creation, deposit, withdrawal, transfer, and closure — were verified to work correctly and consistently, with JDBC transaction handling (commit/rollback) successfully preventing data inconsistencies during fund transfers.

7. My learnings

This internship significantly strengthened my Core Java foundation and gave me practical exposure to how real-world Java applications are designed and built. Key learnings include:

- Object-Oriented Programming — practical application of classes, inheritance, encapsulation, and polymorphism in a real project.
- Exception Handling — using try-catch-finally and custom exceptions for robust error handling.
- Collections Framework — using ArrayList, HashMap, and LinkedList for efficient data management.
- JDBC and SQL — connecting Java applications to MySQL databases, and writing SELECT, INSERT, UPDATE, DELETE, and JOIN queries.
- Design Patterns — applying Singleton and Factory patterns to solve real architectural problems.
- Stream API and Lambda Expressions — writing more concise, readable code for data processing.
- JUnit Testing — writing automated unit tests to validate application correctness.

Beyond technical skills, I also improved my ability to break down a complex project into weekly milestones, collaborate effectively with a team, and incorporate mentor feedback into my design decisions. These skills — technical and interpersonal — will directly support my career growth as a software developer.

8. Future work scope

While the core functionality of the Banking Information System reached 80% completion by the end of the internship, several enhancements are planned or possible for the future:

- Completing the remaining features, along with full end-to-end system testing and UI polish.
- Increasing JUnit test coverage beyond 90%, including integration tests for the JDBC layer.
- Introducing connection pooling (e.g., HikariCP) for more efficient database connection management.
- Applying additional design patterns such as Observer and Strategy for future feature modules.
- Preparing complete project documentation, including a user guide and database schema diagram.
- Extending the console interface to a graphical (GUI) or web-based front end for improved usability.